Optimization Reliability of a Four-Qubit Variational Parity Classifier: Independent Verification of Phases I and II

Field	Value
Project	Quantum Computing Course (claude.ai Project "Quantum Computing")
Author	Dr. Ceasar Jackson Jr.
Date	2026-10-18
Repository	`CeasarJackson/quantum-computing` (checked at local commit `9fc6742`)
Branch	`main`
System under test	PennyLane 0.45.1 four-qubit, 16-parameter variational classifier on 4-bit parity (`labs/PennyLane.py`), its runner `labs/pennylane_repeated_eval.py`, analysis `labs/pennylane_factorial_analysis.py` and Phase II driver `labs/pennylane_phase2_robustness.py`
Studies verified	Phase I factorial study (`docs/pennylane_factorial_study.md`, commit `c8329f5`) and Phase II robustness study (`docs/pennylane_phase2_study.md`, commit `1fc36f7`)

Executive Summary

Phases I and II trained the classifier in 75 unique configurations on disjoint seed sets and reported that 67 reached perfect held-out accuracy. This report recomputes every headline number from the saved result files, re-runs the validation dry runs, and re-executes one failing and one passing configuration from scratch. The recomputed statistics agree with both studies, the failures are all configurations that also failed to fit their training data, and the mechanism of the failures is still not identified. One provenance point in the Phase II study needs a correction: the committed `labs/PennyLane.py` no longer matches the source hash recorded by the Phase II run.

Verified Results

Every row was observed in output produced in this session. "Recomputed" means calculated directly from the saved CSV files with an independent script, not copied from the studies.

#	Check	Result	Evidence
1	Phase I: configurations, mean held-out accuracy, sample SD, perfect count (recomputed from `factorial_records_*.csv`)	25 unique; 94.00%; 20.77 points; 23/25. Matches the study	verification output below
2	Phase I analysis dry run (`pennylane_factorial_analysis.py --dry-run`, canonical CSVs)	PASS: 25 validated, 94.00%, 20.77, 23/25, two warnings for partitions 44 and 45 with init 46	terminal output below
3	Phase II: configurations, mean held-out accuracy, SDs, perfect count (recomputed from `detailed_results.csv`)	50 unique; 90.00%; configuration SD 27.66 (study: 27.7); partition-mean SD 8.82; 44/50. Matches the study	verification output below
4	Phase II mean training accuracy and MSEs	95.33%; 0.122375 and 0.205567 (study: 95.3%, 0.122 / 0.206). Matches	verification output below
5	Training success predicts held-out success	Phase I: 23 with 100% training are the same 23 with 100% held-out, no mixed cases. Phase II: 44 and 44, no mixed cases	verification output below
6	Where failures fall	Phase I: init 46 only (2 of 2). Phase II: init 200 (4) and 203 (2); no partition failed with more than one seed	verification output below
7	Phase II cluster bootstrap over the ten partition means (2000 replicates, seed 20261015)	An independent implementation gives 85.0% to 95.0%. Matches the study's interval	verification output below
8	Phase II classical baselines (means over 50)	Majority 55.0 / 35.0, ridge-linear 60.83 / 20.0, polynomial 100 / 50.0 (abstention-adjusted), nearest neighbour 100 / 0.0 (train / held-out). Matches the study	verification output below
9	Phase II dry run (`pennylane_phase2_robustness.py --dry-run`)	PASS: 10 unique partitions × 5 seeds = 50 runs validated; no files created	terminal output below
10	Re-execution from scratch of partition 101 / init 200 (a recorded failure) and partition 100 / init 200 (a recorded success), 200 steps	Both reproduce the recorded result exactly (differences 0 at the 8 decimals the files store): 101 / 200 again fails (train 66.7%, held-out 0.0%, train MSE 0.88828672, held-	re-execution output below; about 3.5 min

#	Check	Result	Evidence
		out MSE 1.77495892); 100 / 200 again succeeds (100% / 100%, 0.00075120 / 0.00095393). Run on Linux x86_64 (Python 3.13.16, NumPy 2.5.3), not the Mac recorded environment (macOS, NumPy 2.5.4)	per configuration
11	Recorded source hashes against the repository	`labs/PennyLane.py` at HEAD does **not** match the hash recorded by the Phase II run (see Root-Cause Assessment and Evidence Artifact)	`git show` and `git diff` output below

Failure Signature Before Resolution

There was no run-time error; the "failure" is the eight configurations that did not learn parity. They are verbatim from the recomputation (partition, init, training accuracy %, held-out accuracy %, training MSE, held-out MSE):

```
Phase I
  FAIL 44 46 train 66.7 test 25.0 trMSE 0.92563626 teMSE 1.37992014
  FAIL 45 46 train 58.3 test 25.0 trMSE 0.97227313 teMSE 1.19399324
Phase II
  FAIL 101 200 train 66.7 test 0.0 trMSE 0.88828672 teMSE 1.77495892
  FAIL 102 200 train 58.3 test 25.0 trMSE 0.88657331 teMSE 1.48002867
  FAIL 103 200 train 58.3 test 25.0 trMSE 0.94228046 teMSE 1.38103648
  FAIL 105 203 train 66.7 test 0.0 trMSE 0.84237774 teMSE 1.99803367
  FAIL 108 203 train 58.3 test 25.0 trMSE 0.91789917 teMSE 1.62129962
  FAIL 109 200 train 58.3 test 25.0 trMSE 0.95151896 teMSE 1.19235620
```

A training MSE near 0.9 means the circuit's output is near zero or uninformative on the ±1 labels, which is what an unsuccessful optimization looks like.

Controlled Resolution Test

No fix was applied. This section lists what was done so the verification can be reproduced.

1. Copy the saved results and scripts from the repository (`results/pennylane/factorial_analysis/factorial_records_20261015_205050_627551.csv` , `results/`

`pennylane/phase2/20261016T043159_636926Z_41231/detailed_results.csv`, and the five canonical `disjoint_runs_*.csv` files).
2. Recompute the statistics in Verified Results rows 1, 3 to 8 from those CSV files with a short independent script (mean, sample standard deviation, per-partition means, cross-tabulation of training and held-out success, failure counts by seed, and a percentile cluster bootstrap over the ten partition means with `numpy.random.default_rng(20261015)`).
3. Install `pennylane==0.45.1` and run both dry runs: `python3 labs/pennylane_factorial_analysis.py --dry-run` and `python3 labs/pennylane_phase2_robustness.py --dry-run`.
4. Re-execute two configurations from scratch with the Phase I command of Study §6.5: `python3 labs/pennylane_repeated_eval.py --mode initialization --fixed-split-seed <S> --seeds 200 --train-unique 12 --steps 200 --learning-rate 0.1 --ridge 1.0` for S = 101 and S = 100.
5. Compare the recorded source hashes with `git show <commit>:labs/<file> | shasum -a 256` at the commits the Phase II study cites, and at HEAD.

Successful Evidence

Recomputation (excerpt, verbatim):

```
== Phase I: n=25 unique pairs=25
 mean test 94.00  SD(sample) 20.77  perfect test 23/25
 mean train 97.00  mean train MSE 0.096709  mean test MSE 0.127007
 perfect train 23; both perfect 23; train<100 and test==100: 0; train==100 and
test<100: 0
 failures by init {46: 2} by split {44: 1, 45: 1}
== Phase II: n=50 unique pairs=50
 mean test 90.00  SD(sample) 27.66  perfect test 44/50
 mean train 95.33  mean train MSE 0.122375  mean test MSE 0.205567
 perfect train 44; both perfect 44; train<100 and test==100: 0; train==100 and
test<100: 0
 partition-mean SD 8.82 range 80.0 100.0
 failures by init {200: 4, 203: 2} by split {101: 1, 102: 1, 103: 1, 105: 1, 108:
1, 109: 1}
Phase II independent cluster bootstrap 2.5/97.5 pct: [85. 95.]
```

Dry runs, excerpts:

```
Validated 25 unique configurations
Mean held-out accuracy: 94.00%
Held-out sample SD: 20.77 percentage points
Perfect held-out configurations: 23/25
```

```
Split=44 init=46 train=66.7% test=25.0%
Split=45 init=46 train=58.3% test=25.0%
DRY RUN PASS: No analysis exports created

Validated 10 unique partitions × 5 initialization seeds = 50 runs
DRY RUN PASS: no training, files, directories, or logs created
```

Re-execution:

```
split 101 init 200
  quantum_train_mse            new 0.88828672  recorded 0.88828672  diff 0.00e+00
  quantum_train_accuracy_pct   new 66.70000000  recorded 66.70000000  diff 0.00e+00
  quantum_test_mse             new 1.77495892  recorded 1.77495892  diff 0.00e+00
  quantum_test_accuracy_pct    new 0.00000000  recorded 0.00000000  diff 0.00e+00
split 100 init 200
  quantum_train_mse            new 0.00075120  recorded 0.00075120  diff 0.00e+00
  quantum_train_accuracy_pct   new 100.00000000  recorded 100.00000000  diff
0.00e+00
  quantum_test_mse             new 0.00095393  recorded 0.00095393  diff 0.00e+00
  quantum_test_accuracy_pct    new 100.00000000  recorded 100.00000000  diff
0.00e+00
```

Root-Cause Assessment

What the evidence supports:

- The reported results are internally consistent and reproduce from the saved files. All numbers in the two studies that were recomputed here agree (rows 1 to 8).
- Across 75 configurations the split is exact: every configuration that reached 100% training accuracy reached 100% held-out accuracy, and every configuration that missed held-out accuracy had 58–67% training accuracy. No configuration fit its training data and then failed on held-out inputs. This supports reading the failures as unsuccessful optimization within 200 steps rather than overfitting.
- Failures cluster by initialization seed (46 in Phase I; 200 and 203 in Phase II) but no seed fails on every partition, so the outcome depends on the seed-partition combination.
- Successes on all 16 inputs show that the architecture can represent parity on this finite domain. They do not show generalization beyond it or any advantage over classical methods, which the baselines were never designed to test (parity is solvable by a one-line classical rule).

Provenance finding (new, from row 11): the Phase II study (§5.2) says the recorded hash of `labs/PennyLane.py` (`01aca233…`) matches the file at commits `a7730f5` and `c8329f5`, and that a mid-run whitespace-only change was reverted. That is correct for those two commits. However, at the Phase II commit `1fc36f7` and at HEAD the file hashes to `17e0e24b…`, and `git diff a7730f5 HEAD`

shows one removed blank line between two imports and a file-mode change from 100644 to 100755. The study discloses the runner's hash difference but not this one. The difference is whitespace and mode only, so it should not change results, and the re-execution (row 10) ran with the HEAD version.

Experimental limitation

- This report re-ran two of the 75 configurations, not all of them, and the saved Phase II run directory (`split_*_init_*.json` checkpoints, `partition_manifest.json`, `execution.log`) was hashed but not re-compared file by file.
- The Phase I "17 repeated seed combinations" historical audit was not verified: only the five canonical CSVs were available in the sandbox, so the analysis dry run reported 0 historical repeats there.
- The mechanism of the failures (local minima, plateau, or too few steps) is not identified: neither study recorded loss or gradient trajectories, and the Phase II study's remark that seed 46 once escaped a plateau after about 110 steps in earlier course runs was not re-verified here.
- The design has only 16 possible inputs, held-out sets of 4 examples (accuracy moves in 25-point steps), overlapping partitions that are not independent, and five initialization seeds per partition, so the seed clustering is descriptive and no per-seed failure rate can be estimated.
- It is a noiseless simulation; nothing here applies to hardware or noisy circuits. The independent bootstrap used the same seed and replicate count but not necessarily the study's exact resampling code, so agreement is evidence, not proof of identical method.

Recommended Procedure

1. Validate partitions before training (unique inputs, no overlap, full coverage, labels recomputed independently); abort the whole study on any failure.
2. Separate the seeds for the partition and the initialization, and write each configuration's result to its own file with exclusive creation.
3. Commit the code before the run, record the SHA-256 of every source file in the run's configuration, and do not edit those files during the run.
4. After the run, recompute the headline numbers from the raw CSVs with a script that does not share code with the analysis tool.
5. Record loss and gradient norms per step, and re-run the failing configurations at 400–1,000 steps to separate slow convergence from true local minima.
6. When the published code differs from the recorded hashes, say which file and why in the study, and keep the old commit reachable for exact reproduction.

Alignment with Official Documentation

PennyLane documents the gradient method the classifier uses. The `param_shift` page describes the transform as:

> "Transform a circuit to compute the parameter-shift gradient of all gate parameters with respect to its inputs."

and states its assumption: "we assumed that these frequencies are equidistant", pointing to other sources for more general shift rules. (PennyLane, `pennylane.gradients.param_shift`.) The backpropagation tutorial states the compatibility difference between the two gradient methods:

> "`"parameter-shift"` works with all devices (simulator or hardware)."

while `"backprop"` "will only work for specific simulator devices that are designed to support backpropagation", naming `default.qubit` as one. (PennyLane, "Quantum gradients with backpropagation".)

Claim	**Established by**
The parameter-shift transform computes gradients of all gate parameters, and its exactness rests on an equidistant-frequency assumption	PennyLane documentation (`param_shift`)
`"parameter-shift"` works on all devices, `"backprop"` only on specific simulators such as `default.qubit`	PennyLane documentation (backprop tutorial)
That the model's RX/RY/RZ gates satisfy that assumption, so the gradients used for training are exact; and that `--diff-method best` gives the same gradient as parameter-shift	Not established by the pages fetched here. `docs/pennylane.md` asserts both; the Lab 32 test suite is the project's evidence and was not re-run
75 configurations: 67 perfect held-out, and failures coincide exactly with failed training	This session's evidence only: `factorial_records_20261015_205050_627551.csv`, `detailed_results.csv` (SHA-256 below)
The failures are deterministic in the seeds (the first 20 Phase II configurations matched exactly across two executions)	Phase II study §5.1; the re-execution in row 10 tests two of those configurations independently
Parity is not discoverable by the generic classical baselines from 12	

Claim	Established by
examples, and is trivially solvable by a classical rule	The recorded baseline scores (row 8); the rule itself is mathematics, not a documented claim

Note on terminology

"Parameter-shift", `diff_method`, `default.qubit` and "backprop" are PennyLane's terms. "Optimization failure", "held-out accuracy", "partition", "configuration" and "perfect" are this project's operational language. "Failure" here means below 100% held-out accuracy at 200 steps with learning rate 0.1, not a PennyLane error state; the documentation does not define any such criterion.

Evidence Artifact

SHA-256 of the files used, computed on the Mac on 2026-10-18 (`results/` is tracked in the repository):

```
4e7eb520dd88b0ac35c0bf0adffcd4848ac6b075e8e4648aa91690d1aa579043  results/
pennylane/disjoint_runs_20261015_135700_901540.csv
28910845cea8a5713be16c18f3de0c7c17d1221168d6796d2f483a7cd945eef2  results/
pennylane/disjoint_runs_20261015_143226_802847.csv
7a1f2ef7e819100849cabed6b44248cb18095bd1f7d732b4458820ea99cb6e4f  results/
pennylane/disjoint_runs_20261015_194041_859006.csv
51b6e81cabc4149ae898687bf766e84b8b298b4c04e43cdee1386cf8fe7ca094  results/
pennylane/disjoint_runs_20261015_203356_702327.csv
216295fd4cfc4c1b9fec0b251368055090722e2a3fd98336be4b63f116c8b435  results/
pennylane/disjoint_runs_20261015_204308_944440.csv
e50cbc93c68cccd129f7cff78e777bd878b1d57639426b8203f4ac02cefaa351  results/
pennylane/factorial_analysis/factorial_records_20261015_205050_627551.csv
0ebd2f73c5d35f380587c4fc9c1ad7ecebc0fe964ed5555f05e9a9b885020eb6  results/
pennylane/factorial_analysis/factorial_summary_20261015_205050_627551.json
2d53d1c79a0473a8b62722abdb14ec811663e1c679a86d2c92fedb988f9f493d  results/
pennylane/factorial_analysis/historical_occurrences_20261015_205050_627551.json
b4681bbc699349d596f026fd69d65a9407d4d2dd8b42bcc3a58abfc4b9a4d775  results/
pennylane/phase2/20261016T043159_636926Z_41231/configuration.json
d066dd62e674dcbc2961488d2a7423c849211391462c92426506775b26596601  results/
pennylane/phase2/20261016T043159_636926Z_41231/partition_manifest.json
772dbc0f2202082a8025969f0bb6380c1c49bd3e571a08caab1effc5d74a53cc  results/
pennylane/phase2/20261016T043159_636926Z_41231/detailed_results.csv
195dffe80788d9b3eead461bb32cf498018af4d25fe37f620934897e358dd210  results/
pennylane/phase2/20261016T043159_636926Z_41231/partition_statistics.csv
4375acf045414b65300d9f33800ab7b167a581e64cc25b4ffdf45c0cc9e885cb  results/
pennylane/phase2/20261016T043159_636926Z_41231/summary.json
```

```
c044d2696fdadb9f97d00d21de4eecaa4ca1ad7654528ce92964b3fc8c347c2e  results/
pennylane/phase2/20261016T043159_636926Z_41231/execution.log
```

Source files and the hash comparison (Mac, `git show <commit>:<file> | shasum -a 256`):

```
file                              recorded by Phase II run  a7730f5 / c8329f5
1fc36f7 / HEAD
labs/PennyLane.py                 01aca233…cdf9589          01aca233…cdf9589
17e0e24b7ccb912386746a1ca66c51ddc0e2cd6b495ae08f8ac9b766b1d8ff54
labs/pennylane_repeated_eval.py   a3c2781c…097749d          a3c2781c…097749d
19b606036847688cd6e9e547efffc8bfa15b3ddb9f1e2b5057dd652fdd3b7cd6
```

Working-tree hashes at HEAD: `labs/pennylane_phase2_robustness.py` `5912ddf0…6a465d9`, `labs/pennylane_factorial_analysis.py` `71a653c0…b162f`. The recomputation script was an ad-hoc session script (not committed); its logic is the list in Controlled Resolution Test step 2 and its full output is quoted above.

Portfolio Significance

This work shows verification as a separate step from reporting: results that had already been written up were recomputed from the raw files with an independent script, the guard checks were re-run, and two configurations were re-executed from scratch. It also shows the discipline of reporting what that check turned up, here a provenance mismatch in the study's own disclosure, rather than only confirming the agreeable numbers. The experimental design itself (validated partitions, separated seeds, exclusive-create result files, recorded source hashes, bootstrap over clusters rather than runs) is the kind of data-engineering practice that makes such a check possible. The honest end state is a reproducible record of optimization reliability for a small model, with the failure mechanism still open and a clear next experiment.

References

1. PennyLane. "pennylane.gradients.param_shift." https://docs.pennylane.ai/en/stable/code/api/pennylane.gradients.param_shift.html
2. PennyLane. "Quantum gradients with backpropagation." https://pennylane.ai/demos/tutorial_backprop
3. Repository `CeasarJackson/quantum-computing`: `docs/pennylane_factorial_study.md`, `docs/pennylane_phase2_study.md`, `docs/pennylane.md`.